UNIVERSITÀ
DI TORINO

Università degli Studi di Torino

Corso di Laurea Magistrale in Fisica dei Sistemi Complessi

Improving VAE Representational Capabilities for Understanding LLMs

Tesi di Laurea Magistrale

Relatore:
Prof. Michele Caselle

Corelatori:
Prof. Alan Perotti
Prof. André Panisson

Controrelatore:
Prof. Matteo Osella

Candidato:
Francesco Marchisotti

Tutor Aziendale:
Gabriele Ciravegna

Anno Accademico 2025/2026

[6th June 2026 at 15:33 – classicthesis v4.8]

Francesco Marchisotti: *Improving VAE Representational Capabilities for Understanding LLMs,* Fisica dei Sistemi Complessi

CENTAI TEAM:
Gabriele Ciravegna
Alan Perotti
André Panisson
Francesco Cozzi

UNITO PROFESSORS:
Michele Caselle
Matteo Osella

LOCATION:
Torino

TIME FRAME:
Oct. 2025 - Jul. 2026

"I wish there was a way to know you're in the good old days
before you've actually left them."

— *Andy Bernard*

ABSTRACT

Short summary of the contents in English. . . a great guide by Kent Beck how to write good abstracts can be found here:

https://plg.uwaterloo.ca/~migod/research/beckOOPSLA.html

ZUSAMMENFASSUNG

Kurze Zusammenfassung des Inhaltes in deutscher Sprache. . .

ACKNOWLEDGMENTS

To whom shall be acknowledged,
you are being acknowledged with this exact statement.

"I'm so funny."

— *The candidate, who's not actually funny*

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LISTINGS

ACRONYMS

1

INTRODUCTION

1.1 MOTIVATION

1.2 THESIS STRUCTURE

2

BACKGROUND AND RELATED WORK

2.1 NLP

2.2 XAI

2.2.1 *Traditional XAI*

2.2.2 *C-XAI*

2.2.2.1 *Explainable-by-design Models*

CBM / CEM

LF-CBM

2.2.2.2 *Post-hoc Methods*

T-CAV

2.3 AUTOENCODERS

An AutoEncoder (AE) is a neural network trained to reconstruct its input through an intermediate latent representation [5, Ch 14]. It consists of two components: an **encoder** $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$ that maps an input $\mathbf{x}$ to a latent code $\mathbf{z} = f_\theta(\mathbf{x})$, and a **decoder** $g_\phi : \mathbb{R}^m \to \mathbb{R}^n$ that reconstructs the input as $\hat{\mathbf{x}} = g_\phi(\mathbf{z})$. Training minimises a reconstruction loss, typically the Mean Squared Error:

$$\mathcal{L}_{\text{recon}} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2.$$

When $m < n$, the bottleneck forces the model to learn a compressed representation. When $m > n$, the network is overcomplete and additional constraints are required to prevent the trivial solution of learning the identity. In either case, the structure of the latent space is shaped by the choice of architecture and regularisation.

2.3.1 *Usage*

IMAGE RETRIEVAL Krizhevsky and Hinton [7] proposed using deep autoencoders for content-based image retrieval. Their key insight is that a deep autoencoder can learn to map images to short binary codes that preserve semantic similarity: images that are perceptually similar

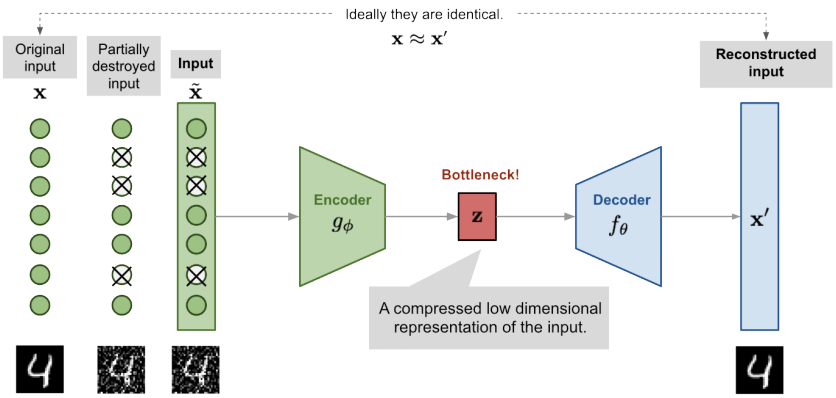

Figure 2.1: Illustration of Denoising AutoEncoder model architecture, reproduced from Weng [16]. *Note: this figure uses the convention f_θ for the decoder and g_ϕ for the encoder, which is the reverse of the notation adopted in this thesis.*

are mapped to nearby codes in the latent space, allowing retrieval to be reduced to a fast lookup rather than an exhaustive search over the full database. They demonstrate this on a dataset of 1.6 million tiny images, showing that codes produced by deep autoencoders yield qualitatively and quantitatively better retrieval results than matching in raw pixel space. This work illustrated that the compressed latent representation learned by an autoencoder need not merely serve reconstruction, but can itself be a semantically meaningful and practically useful object.

DENOISING AUTOENCODER This variation was introduced by Vincent et al. [15] to avoid overfitting and improve the robustness of the learned representation. The input is partially corrupted by adding noise to or masking some values of the input vector in a stochastic manner, $\mathbf{x}_c \sim \mathcal{M}_\mathcal{D}(\mathbf{x}_c \mid \mathbf{x})$, as shown in fig. 2.1. The model is then trained to reconstruct the original uncorrupted input:

$$\mathcal{L}_{\text{DAE}} = \left\| \mathbf{x} - g_\phi(f_\theta(\mathbf{x}_c)) \right\|_2^2.$$

This design is motivated by the fact that humans can easily recognize an object or a scene even the view is partially occluded or corrupted. To "repair" the partially destroyed input, the denoising autoencoder has to discover and capture relationship between dimensions of input in order to infer missing pieces.

For high dimensional input with high redundancy, like images, the model is likely to depend on evidence gathered from a combination of many input dimensions to recover the denoised version rather than to overfit one dimension. This builds up a good foundation for learning *robust* latent representation.

The noise is controlled by a stochastic mapping $\mathcal{M}_\mathcal{D}(\mathbf{x}_c \mid \mathbf{x})$, and it is not specific to a particular type of corruption process (i. e., mask-

ing noise, Gaussian noise, salt-and-pepper noise, etc.). Naturally the corruption process can be equipped with prior knowledge [16].

ANOMALY DETECTION Sakurada and Yairi [12] were among the first to explicitly propose autoencoders as a tool for anomaly detection. Their core insight is that an autoencoder trained on normal data learns a compressed representation of the normal data distribution; at test time, inputs that deviate from this distribution are reconstructed poorly, and the resulting reconstruction error serves as an anomaly score. To substantiate this claim, they apply autoencoders to both synthetic data generated from the Lorenz system and real spacecraft telemetry, comparing against linear PCA and kernel PCA. They demonstrate that autoencoders, by virtue of their nonlinear dimensionality reduction, are able to detect subtle anomalies that linear PCA misses, while remaining computationally lighter than kernel PCA. This work established reconstruction error as a natural unsupervised anomaly signal for autoencoder-based methods.

2.3.2 *Sparse AutoEncoders*

Sparse AutoEncoders (SAEs) are one of the main variants of AE. Just like AEs, they learn a dictionary of feature directions from the input data, but they tipically are overcomplete ($m \gg n$) and their latent representation is forced to be sparse using some kind of sparsity mechanism. Given input $\mathbf{x} \in \mathbb{R}^n$, the SAE encodes it as:

$$\mathbf{z} = \mathrm{ReLU}(\mathbf{W}_e(\mathbf{x} - \mathbf{b}_{\text{pre}}) + \mathbf{b}_e),$$

where $\mathbf{W}_e \in \mathbb{R}^{m \times n}$ is the encoder weight matrix, $\mathbf{b}_e \in \mathbb{R}^m$ is the encoder bias, and $\mathbf{b}_{\text{pre}} \in \mathbb{R}^n$ is a pre-encoder bias subtracted from the input before encoding. This learned offset allows the encoder to operate on roughly mean-centered activations, improving training stability [2]. The decoder then reconstructs the input as:

$$\hat{\mathbf{x}} = \mathbf{W}_d \mathbf{z} + \mathbf{b}_{\text{pre}},$$

where $\mathbf{W}_d \in \mathbb{R}^{n \times m}$ is the decoder weight matrix. Each column $\mathbf{d}_i$ of $\mathbf{W}_d$ is a dictionary atom, representing a candidate feature direction in activation space.

When used in the context of Mechanistic Interpretability (MechInt), the overcompleteness and the sparsity constraint are motivated by the *superposition hypothesis*.

2.3.2.1 *The Superposition Hypothesis*

A central challenge in MechInt is that individual neurons in neural networks tend to be *polysemantic*: they activate in response to multiple, seemingly unrelated concepts [3, 8]. An example of this can be seen in

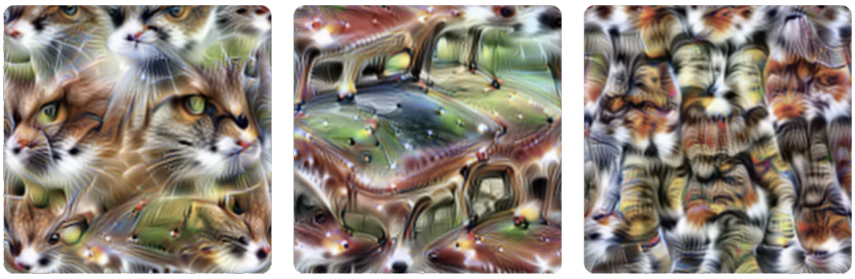

Figure 2.2: Neuron 4e:55, a polysemantic neuron from InceptionV1 which responds to cat faces, fronts of cars, and cat legs [8].

fig. 2.2. This runs contrary to the intuition that an interpretable model should have neurons with clean, well-defined roles.

Elhage et al. [3] proposed the *superposition hypothesis* to explain this phenomenon. The core idea is that a network may represent more features than it has dimensions, by exploiting the fact that natural inputs are sparse: at any given time, only a small fraction of all possible features are relevant. Under this condition, a network can pack many near-orthogonal feature directions into a lower-dimensional space with limited interference; polysemanticity ceases to be a failure of the model, and instead becomes a rational compression strategy under capacity constraints.

This has a direct implication for interpretability: the natural basis of a model's activations is not the neuron basis, but a larger, overcomplete set of feature directions embedded within it. Recovering this basis requires moving beyond individual neuron analysis, towards methods that can identify sparse, distributed structure. This is precisely the goal of *sparse dictionary learning* [9], a classical technique from computational neuroscience that SAEs adapt to the setting of modern language models.

2.3.2.2 *Architectures*

There are many ways to enforce sparsity, each with its pros and cons. Here is a selection of methods that range from the initial SAE idea to more modern architectures that try to improve on it.

VANILLA ℓ_1 This is the original architecture proposed by Huben et al. [6]. It uses an ℓ_1[1] penalty on the latent code to encourage most of $\mathbf{z}$'s entries to be exactly zero, so that any given activation is reconstructed from a small number of active features. The loss used to train this model is the following:

$$\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \lambda \|\mathbf{z}\|_1.$$

The scalar λ controls the sparsity–reconstruction trade-off and must be tuned carefully: too large, and reconstruction degrades; too small,

1 The ℓ_p norm is defined as $\ell_p(\mathbf{x}) = \|\mathbf{x}\|_p = \left(\sum_i |x_i|^p\right)^{1/p}$, $p \geq 1$.

and the learned features lose monosemanticity. A known side effect of ℓ_1 regularisation is *shrinkage* [14, 17]: beyond driving inactive features to zero, the penalty also biases active feature magnitudes below their reconstruction-optimal values, causing the model to systematically underestimate the true activation of each feature [10, 17]. Constraining the decoder columns to unit ℓ_2 norm after each gradient step partially mitigates this, by preventing the model from reducing the ℓ_1 penalty through the trivial reparameterisation of scaling down encoder outputs and compensating with larger decoder norms [10]. There is, however, a residual gradient-level bias that is structurally entangled with the ℓ_1 penalty and can't be resolved with this trick.

TOPK SAES Gao et al. [4] proposed replacing the ℓ_1 penalty with a hard sparsity constraint, implemented via a TopK activation function that retains only the k largest pre-activations and zeros the rest. This directly controls the number of active features per forward pass, eliminating the need to tune λ and removing the shrinkage bias entirely. The full training objective is:

$$\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \alpha \mathcal{L}_{\text{aux}},$$

where the auxiliary loss $\mathcal{L}_{\text{aux}}$ addresses the problem of dead latents, i. e., features that cease to activate entirely during training. Latents are flagged as dead if they have not activated for a predetermined number of tokens. Given the reconstruction residual $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$, the top-$k_{\text{aux}}$ dead latents are selected and used to reconstruct $\mathbf{e}$; the resulting reconstruction error is $\mathcal{L}_{\text{aux}}$. This gives dead latents a gradient signal proportional to how much of the residual they could explain, encouraging them to activate rather than remain permanently dormant. Gao et al. [4] demonstrate that TopK SAEs achieve better reconstruction at equivalent sparsity levels compared to ℓ_1 baselines, making them a strong and widely used alternative.

GATED SAES Rajamanoharan et al. [10] introduced an architecture that separates the decision of whether a feature is active from the decision of how strongly it activates. A dedicated gating pathway produces a binary mask via a Heaviside step function (approximated during training with a straight-through estimator) which is applied element-wise to the encoder output. This architectural separation allows the model to learn feature presence and magnitude more independently, addressing shrinkage from a structural rather than a loss-based perspective.

JUMPRELU SAES Rajamanoharan et al. [11] proposed replacing the standard ReLU with a JumpReLU activation, which is zero below a learned per-feature threshold θ_i and equal to the pre-activation above it:

$$\text{JumpReLU}_{\theta_i}(z_i) = z_i \cdot \mathbb{1}[z_i > \theta_i].$$

The key advantage of this parameterisation is that it provides a dedicated, per-feature learnable threshold, which allows the ℓ_0 norm of the feature activations — $\sum_i H(\pi_i - \theta_i)$, where π_i are the encoder pre-activations and $H(x) = \mathbb{1}[x > 0]$ is the Heaviside step function — to be trained directly via a straight-through estimator applied to the threshold parameter θ_i. Because each threshold can be positioned near the actual boundary between active and inactive pre-activations for its feature, the gradient signal is stable and controllable, making ℓ_0 training practical in a way that would not be possible with a fixed threshold. Training with ℓ_0 rather than ℓ_1 eliminates shrinkage entirely, since ℓ_0 penalises only the number of active features and not their intensity. JumpReLU SAEs achieve performance competitive with TopK SAEs.

2.3.2.3 *Application to Mechanistic Interpretability*

The primary application of SAEs in MechInt is *feature discovery*: training a SAE on the internal activations of a language model and examining the resulting dictionary atoms to identify human-interpretable concepts. The underlying hypothesis is that each dictionary atom, being active for only a semantically coherent set of inputs, corresponds to a monosemantic feature of the model's computation, a clean unit of meaning that the model had obscured in the neuron basis.

Make sure to mention what the residual stream, MLP, and attention heads are in a chapter about LLMs

Do we have a reference for this?

WHERE SAES ARE APPLIED SAEs can be trained on activations from any layer or component of a transformer. Common targets include the residual stream at various depths, MLP layer outputs, and attention head outputs [2, 6]. Each choice decomposes a different aspect of the model's representations, and the resulting features reflect the computational role of the targeted component. Residual stream SAEs, for instance, tend to capture more semantic, higher-level structure, while MLP-targeted SAEs often recover features more directly tied to knowledge retrieval and factual associations.

FEATURE INTERPRETABILITY In practice, learned features often correspond to recognisable concepts. Bricken et al. [2] trained a SAE on the MLP of a one-layer transformer and found features responsive to specific tokens, syntactic roles, and semantic categories. Scaling this approach to production-scale models, Templeton et al. [13] claim to have identified lots of interpretable features, including neurons representing famous people or countries, emotions, or even biases (racism, sexism, hatred) and sarcastic praise. These results provide strong empirical support for the superposition hypothesis and suggest that SAEs can serve as a practical lens through which to study what information LLMs represent.

EVALUATION Assessing the quality of learned features is non-trivial, as interpretability is inherently subjective. Several proxy metrics are used in practice. *Reconstruction quality* is typically measured by the downstream Cross-Entropy loss recovered when replacing original activations with SAE reconstructions [4]. *Sparsity* is quantified via the average ℓ_0 norm of the latent codes, i. e., the mean number of active features per token. *Automated interpretability* pipelines use a second language model to generate and score natural-language descriptions of individual features [1], providing a scalable if imperfect proxy for human judgement.

Could this be made clearer?

The inherent tension between these metrics – reconstruction quality, sparsity, and interpretability – is a recurring theme in SAE literature, and motivates both the architectural variants described in section 2.3.2.2 and the alternative approach developed in this thesis and described in section 3.3.

2.3.3 *VAEs*

2.3.3.1 *Architectures*

β-VAE

VQ-VAE

2.3.3.2 *Usage*

GENERATIVE MODEL

XAI

CONCLUSIONS

3 METHODS

4 EXPERIMENTS

5 CONCLUSIONS

5.1 CONTRIBUTIONS

5.2 FUTURE WORK

BIBLIOGRAPHY

[1] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu and William Saunders. 'Language Models Can Explain Neurons in Language Models'. In: *OpenAI* (May 2023). (Visited on 29/05/2026).

[2] Trenton Bricken et al. 'Towards Monosemanticity: Decomposing Language Models With Dictionary Learning'. In: *Transformer Circuits Thread* (Oct. 2023). (Visited on 28/05/2026).

[3] Nelson Elhage et al. 'Toy Models of Superposition'. In: *Transformer Circuits Thread* (Sept. 2022). (Visited on 29/05/2026).

[4] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike and Jeffrey Wu. 'Scaling and Evaluating Sparse Autoencoders'. In: *The Thirteenth International Conference on Learning Representations*. Oct. 2024. (Visited on 14/05/2026).

[5] Ian Goodfellow, Aaron Courville and Yoshua Bengio. *Deep Learning*. Adaptive Computation and Machine Learning. Cambridge, Massachusetts: The MIT Press, 2016. ISBN: 978-0-262-03561-3 978-0-262-33737-3.

[6] Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart and Lee Sharkey. 'Sparse Autoencoders Find Highly Interpretable Features in Language Models'. In: *The Twelfth International Conference on Learning Representations*. Oct. 2023. (Visited on 28/05/2026).

[7] Alex Krizhevsky and Geoffrey E. Hinton. 'Using Very Deep Autoencoders for Content-Based Image Retrieval'. In: *19th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning* (Apr. 2011).

[8] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov and Shan Carter. 'Zoom In: An Introduction to Circuits'. In: *Distill* 5.3 (Mar. 2020), e00024.001. ISSN: 2476-0757. DOI: 10.23915/distill.00024.001. (Visited on 04/05/2026).

[9] Bruno A. Olshausen and David J. Field. 'Sparse Coding with an Overcomplete Basis Set: A Strategy Employed by V1?' In: *Vision Research* 37.23 (Dec. 1997), pp. 3311–3325. ISSN: 0042-6989. DOI: 10.1016/S0042-6989(97)00169-7. (Visited on 28/05/2026).

[10] Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah and Neel Nanda. 'Improving Sparse Decomposition of Language Model Activations with Gated Sparse Autoencoders'. In: *Advances in Neural Information Processing Systems* 37 (Dec. 2024), pp. 775–818. DOI: 10.52202/079017-0024. (Visited on 05/06/2026).

[11] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár and Neel Nanda. *Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders*. Aug. 2024. DOI: 10.48550/arXiv.2407.14435. arXiv: 2407.14435 [cs]. (Visited on 23/02/2026).

[12] Mayu Sakurada and Takehisa Yairi. 'Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction'. In: *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*. MLSDA'14. New York, NY, USA: Association for Computing Machinery, Dec. 2014, pp. 4–11. ISBN: 978-1-4503-3159-3. DOI: 10.1145/2689746.2689747. (Visited on 05/06/2026).

[13] Adly Templeton et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html#appendix-citation. (Visited on 20/10/2025).

[14] Robert Tibshirani. 'Regression Shrinkage and Selection Via the Lasso'. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 58.1 (Jan. 1996), pp. 267–288. ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1996.tb02080.x. (Visited on 29/05/2026).

[15] Pascal Vincent, Hugo Larochelle, Yoshua Bengio and Pierre-Antoine Manzagol. 'Extracting and Composing Robust Features with Denoising Autoencoders'. In: *Proceedings of the 25th International Conference on Machine Learning - ICML '08*. Helsinki, Finland: ACM Press, 2008, pp. 1096–1103. ISBN: 978-1-60558-205-4. DOI: 10.1145/1390156.1390294. (Visited on 05/06/2026).

[16] Lilian Weng. *From Autoencoder to Beta-VAE*. Aug. 2018. (Visited on 05/06/2026).

[17] Benjamin Wright and Lee Sharkey. 'Addressing Feature Suppression in SAEs — AI Alignment Forum'. In: (Feb. 2024). (Visited on 29/05/2026).

COLOPHON

This document was typeset using the typographical look-and-feel `classicthesis` developed by André Miede and Ivo Pletikosić. The style was inspired by Robert Bringhurst's seminal book on typography "*The Elements of Typographic Style*". `classicthesis` is available for both LaTeX and LyX:

https://bitbucket.org/amiede/classicthesis/

Happy users of `classicthesis` usually send a real postcard to the author, a collection of postcards received so far is featured here:

http://postcards.miede.de/

Thank you very much for your feedback and contribution.